

ESTRATEGIA Y PROTOCOLO DE CONTROL DE VERSIONES (GIT WORKFLOW)

Convención de Ramas, Reglas de Pull Request y Estándar Conventional Commits — Normas APA (7.ª Ed.)

LÍDER DE ARQUITECTURA & QA
Alan Martinez

FECHA Y VERSIÓN
2026-08-25 | Versión 2.0
(Estrategia Aprobada)

EQUIPO DE DESARROLLO
I. Cardozo, F. Sarraute, A. Heredia,
M. Silvani

MODELO DE TRABAJO GIT
Gitflow Adaptado + Conventional
Commits

1. PROPÓSITO Y MODELO DE TRABAJO

El presente documento fija el protocolo obligatorio de gobernanza de código fuente para el repositorio del proyecto **Plataforma Hospitalaria de Código Azul**. Se adopta un modelo basado en **Gitflow Adaptado** para garantizar la estabilidad del código, integración continua fluida y trazabilidad de cambios en equipo.

2. ESTRUCTURA Y CONVENCIÓN DE NOMBRES PARA RAMAS

Queda estrictamente prohibido realizar commits directos sobre las ramas principales (main / develop). Todo cambio debe realizarse a través de ramas auxiliares debidamente tipadas:

Tabla 1. Tipología de Ramas y Políticas de Fusión

TIPO DE RAMA	PATRÓN DE NOMBRE	PROPÓSITO TÉCNICO Y RAMA DE DESTINO
Main (Producción)	main	Código estable, probado y listo para evaluación. Protegida contra pushes directos.
Development	develop	Rama de integración continua. Fusiona las funcionalidades listas antes del release.
Feature	feature/<módulo>-<descripcion>	Desarrollo de nuevas funcionalidades (ej. feature/fcm-token-registry). Nace de develop, integra a develop.
Bugfix	bugfix/<módulo>-<descripcion>	Corrección de defectos encontrados en etapa de QA. Nace de develop, integra a develop.
Hotfix	hotfix/<descripcion-critica>	Parche de emergencia para fallos críticos en producción. Nace de main, integra a main y develop.

REGLA DE AISLAMIENTO FEATURE-FIRST

Cada desarrollador trabaja en la subcarpeta de su módulo funcional (src/features/auth/, src/features/codigo_azul/, src/features/fcm/, client/) reduciendo al mínimo la probabilidad de conflictos de fusión (merge conflicts).

3. CONVENCION DE COMMITS (CONVENTIONAL COMMITS V1.0.0)

Todos los mensajes de commit deben seguir estrictamente el estándar:

```
<tipo>(<alcance/módulo>): <descripción corta en presente y minúsculas>
```

Ejemplos reales aplicados en el repositorio del proyecto:

- feat(auth): implementar middleware de validación de token JWT y roles RBAC
- feat(fcm): añadir use-case de despacho push con Doze Mode bypass y auditoría JSONB
- fix(db): corregir trigger append-only fn_prevent_audit_tampering en PostgreSQL 18
- test(e2e): incorporar tests 13 a 17 para validación de ciclo de vida de tokens FCM
- refactor(express): migrar configuración del router al estándar nativo de Express 5

4. REGLAS DE PULL REQUESTS (PR) Y REVISIÓN DE CÓDIGO

Para fusionar cambios en develop o main es obligatorio cumplir con las siguientes políticas de calidad:

- **Revisión de Código (Code Review):** Aprobación técnica obligatoria por el Líder de Arquitectura (Alan Martinez) o revisor asignado.
- **Verificación de Suite Automatizada:** Ninguna PR se fusiona si no supera el 100% de la suite de pruebas (**17/17 tests passing**).
- **Estrategia de Merge:** *Squash and Merge* obligatorio para mantener un árbol de historial limpio y lineal.

5. REFERENCIAS NORMATIVAS (APA 7.ª EDICIÓN)

Chacon, S., & Straub, B. (2014). *Pro Git* (2.ª ed.). Apress.

Conventional Commits. (2020). *Conventional Commits specification* (Version 1.0.0). <https://www.conventionalcommits.org/>

IEEE Computer Society. (2011). *IEEE Standard for Configuration Management in Systems and Software Engineering* (IEEE Std 828-2012). IEEE.